

QUESTÕES

Questão 1

An HTML form contains this form element:

```
<input type="file" name="myFile">
```

When this form is submitted, the following PHP code gets run:

```
move_uploaded_file(
    $_FILES['myFile']['tmp_name'],
    'uploads/' . $_FILES['myFile']['name']
);
```

Alternativas:

- a) Check the charset encoding of the HTTP request to see whether it matches the encoding of the uploaded file.
- b) Use \$HTTP_POST_FILES instead of \$_FILES to maintain upwards compatibility.
- c) Check with is_uploaded_file() whether the uploaded file \$_FILES['myFile']['tmp_name'] is valid.
- d) Sanitize the file name in \$_FILES['myFile']['name'] because this value could be forged.

Questão 2

Given the following code:

```
namespace PHP;

enum Category {
    case Info;
    case Warning;
    case Error;
};

function log(string $message, Category $category) {
    echo "$message {$category->name}";
}

log("My message", ***);
```

Which expression may be used instead of the `***` placeholder to log an "Info" message?

Alternativas:

- a) `Category['Info']`
- b) `Category.Info`
- c) `Category::Info`
- d) `Category->Info`

Questão 3

How can you create an HTTP POST request in PHP without having to install or activate a PHP extension?

Alternativas:

- a) `file_get_contents()`
- b) `curl_exec()`
- c) PHP only supports GET requests
- d) `http_build_query()`

Questão 4

Which of these approaches is recommended to protect a web site against Cross-Site Request Forgery (CSRF)? (Choose 2)

Alternativas:

- a) Use the SameSite flag for cookies.
- b) Add an additional token to the HTTP request.
- c) Validate the IP address of the user.
- d) Use POST instead of GET.
- e) Disable sessions.

Questão 5

Which of the following tasks can be achieved by using magic methods? (Choose 3)

Alternativas:

- a) Creating an iterable object.
- b) Processing access to undefined methods or properties.
- c) Initializing or uninitializing object data.
- d) Creating a new stream wrapper.
- e) Converting objects to string representation.
- f) Overloading operators like +, *, etc.

Questão 6

What is the output of the following code?

```
namespace MyNamespace;  
  
class Test {  
}  
  
echo Test::class;
```

Alternativas:

- a) parse error
- b) empty string
- c) MyNamespace\Test
- d) Test

Questão 7

Which of the following is a package manager that allows adding dependencies to a PHP project?

Alternativas:

- a) PHAR
- b) Composer
- c) Maven
- d) TAR

Questão 8

Your User class has a \$role property. All code working with an instance of that class shall be able to retrieve the property's value, but only the class itself must be allowed to change it. Which of the following code snippets satisfies these requirements?

Alternativas:

a)

```
class User
{
    public private(set) string $role;
}
```

b)

```
class User
{
    private public(get) string $role;
}
```

c)

```
class User
{
    public private string $role;
}
```

d)

```
class User
{
    public(get) private(set) string $role;
}
```

Questão 9

What is the benefit of using persistent database connections in PHP? (Choose 2)

Alternativas:

- a) Allows connection settings such as character set encoding to be remembered after a PHP script ends.
- b) Allows for resumption of transactions across multiple requests.
- c) Ensures that only a single connection is open to the database from PHP.
- d) Reduces the connection & authentication overhead of connecting to the database.

Questão 10

Which regular expression will match each of these strings?

(000) 555-1234
000 555 1234
000-555-1234
000.555.1234

Alternativas:

- a) `/^\.*\d{3}.\d{3}.\d{4}$/`
- b) `/^(\d{3}).\d{3}.\d{4}$/`
- c) `/^\.*(\d{3}.\d{3}.\d{4})$/`
- d) `/^(\d{3}).\d{3}.\d{4}$/`

Questão 11

Table data (table name "users" with primary key "id"):

id	name	email
1	anna	alpha@example.com
2	betty	beta@example.org
3	clara	gamma@example.net
5	siggi	sigma@example.info

PHP code (assume the PDO connection is correctly established):

```
$dsn = 'mysql:host=localhost;dbname=exam';
$user = 'username';
$pass = '*****';
$pdo = new PDO($dsn, $user, $pass);

$cmd = "SELECT name, email FROM users LIMIT 1";
$stmt = $pdo->prepare($cmd);
$stmt->execute();

$result = $stmt->fetchAll(PDO::FETCH_BOTH);
$row = $result[0];
```

Alternativas:

- a) The value of \$row is array(0 => 'anna', 'name' => 'anna', 1 => 'alpha@example.com', 'email' => 'alpha@example.com').
- b) The value of \$row is array('name' => 'anna', 'email' => 'alpha@example.com').
- c) The value of \$row is array(0 => 'anna', 1 => 'alpha@example.com').
- d) The value of \$result is array('anna' => 'alpha@example.com').

Questão 12

What is the output of the following code?

```
$a = ['a', 'b' => 'c'];  
echo property_exists((object) $a, 'a') ? 'true' : 'false';  
echo '-';  
echo property_exists((object) $a, 'b') ? 'true' : 'false';
```

Alternativas:

- a) false-false
- b) true-false
- c) true-true
- d) false-true

Questão 13

Which of the following PHP functions can be used to set the HTTP response code?
(Choose 2)

Alternativas:

- a) `http_response_code()`
- b) `header_add()`
- c) `http_set_status()`
- d) `http_header_set()`
- e) `header()`

Questão 14

You want to work with a class where you can set arbitrary properties. You end up with the following code:

```
class Person {  
}  
  
$person = new Person();  
$person->firstName = "Jane";  
$person->lastName = "Doe";
```

When running this code, you notice that you get a deprecation notice on some systems, and you are anticipating that this code might not work at all in a future PHP version.

Which of these approaches does **NOT** allow you to fulfill your requirements while getting rid of the deprecation messages?

Alternativas:

a) Replace the class implementation with:

```
dynamic class Person {  
}
```

b) Replace the class implementation with:

```
$person = new stdClass();  
$person->firstName = "Jane";  
$person->lastName = "Doe";
```

c) Replace the class implementation with:

```
class Person {  
    protected $properties = [];  
  
    public function __get($key) {  
        return $this->properties[$key] ?? null;  
    }  
  
    public function __set($key, $value) {  
        $this->properties[$key] = $value;  
    }  
}
```

d) Replace the class implementation with:

```
#[\AllowDynamicProperties]  
class Person {  
}
```


Questão 15

How can the `id` attribute of the second paragraph element from the XML string below be retrieved from the `SimpleXML` object found inside `$xml`?

```
<?xml version="1.0"?>
<document>
  <section>
    <paragraph id="1">One</paragraph>
    <paragraph id="2">Two</paragraph>
  </section>
</document>
```

Alternativas:

a)

```
$xml->foo->section->paragraph[1]['id']
```

b)

```
$xml->document->paragraph[2]['id']
```

c)

```
$xml->getElementById('2');
```

d)

```
$xml->document->section->paragraph[2]['id']
```

e)

```
$xml->section->paragraph[1]['id']
```

Questão 16

What is the output of the following code?

```
function increment ($val)
{
    $val = $val + 1;
}

$val = 1;

increment($val);

echo $val;
```

Questão 17

What is the output of the following code?

```
$a = [1, 2, 3] + [4, 5, 6, 7];  
echo implode(' ', $a);
```

Alternativas:

- a) 1, 2, 3, 4, 5, 6, 7
- b) 1, 2, 3, 7
- c) 4, 5, 6, 7
- d) 5, 7, 9, 7

Questão 18

What is the output of the following code?

```
$rnd = rand(1, 10);  
  
$category = match (true) {  
    $rnd <= 10 => 'high',  
    $rnd <= 6  => 'mid',  
    $rnd < 3   => 'low'  
};  
  
echo $category;
```

Alternativas:

- a) Always 'high'
- b) Always 'low'
- c) Fatal error
- d) Either 'high' or 'mid' or 'low', determined at random

Questão 19

Which of the following expressions will evaluate to a random value from an array below?

```
$array = ["Sue","Mary","John","Anna"];
```

Alternativas:

a)

```
$array[array_rand($array)];
```

b)

```
array_values($array, ARRAY_RANDOM);
```

c)

```
shuffle($array);
```

d)

```
array_rand($array);
```

e)

```
array_rand($array, 1);
```

Questão 20

What is the most general PHP type to use in a catch block in order to catch all errors and exceptions?

Alternativas:

- a) Exception
- b) Error
- c) Throwable
- d) RuntimeException

Questão 21

What is the result of the following bitwise operation in PHP?

```
1 ^ 2
```

Alternativas:

- a) -1
- b) 3
- c) 4
- d) 2
- e) 1

Questão 22

What purpose do namespaces fulfill?

Alternativas:

- a) Alternative to classes.
- b) Improved performance.
- c) Encapsulation.
- d) All of the above.

Questão 23

What is the output of the following code?

```
$text = 'This is text';  
  
$text1 = <<<TEXT  
$text  
TEXT;  
  
$text2 = <<<TEXT  
$text1  
TEXT;  
  
echo "$text2";
```

Alternativas:

- a) \$text2
- b) \$text1
- c) \$text
- d) This is text

Questão 24

What is the output of the following code?

```
class a
{
    public $val;
}

function renderVal (a $a)
{
    if ($a) {
        echo $a->val;
    }
}

renderVal(null);
```

Alternativas:

- a) An error, because null is not an instance of a.
- b) A syntax error in the function declaration line.
- c) Nothing, because a null value is being passed to renderVal().
- d) NULL

Questão 25

Which of these code snippets **DO NOT** write the exact content of the file `source.txt` to `target.txt`? (Choose 2)

Alternativas:

a)

```
$handle = fopen("target.txt", "w+");  
fwrite($handle, file_get_contents("source.txt"));  
fclose($handle);
```

b)

```
file_put_contents("target.txt", join(file("source.txt"), "\n"));
```

c)

```
file_put_contents("target.txt", fopen("source.txt", "r"));
```

d)

```
file_put_contents("target.txt", readfile("source.txt"));
```

e)

```
file_put_contents("target.txt",  
file_get_contents("source.txt"));
```

Questão 26

Is the following code vulnerable to SQL Injection?

```
// assume $pdo is a valid PDO connection
$age = $pdo->quote((int) $age);
$name = $pdo->quote($name);
$query = "SELECT * FROM users WHERE name LIKE
'$name' AND age = $age";
$result = $pdo->query($query);
```

Alternativas:

- a) Yes, because the \$name variable is improperly escaped.
- b) Yes, because even though \$age and \$name are escaped, nothing prevents their contents from modifying the SQL.
- c) Yes, because the \$age variable is improperly escaped.
- d) No, the code is fully protected from SQL Injection.
- e) Yes, because you cannot prevent SQL Injection when using PDO.

Questão 27

Which of the following does **NOT** help to protect against session hijacking and fixation attacks?

Alternativas:

- a) Protect against XSS vulnerabilities in the application.
- b) Use HTTPS and set the secure cookie flag.
- c) Set the `session.use_only_cookies` php.ini parameter to 1.
- d) Set the `session.cookie_lifetime` php.ini parameter to 0.
- e) Rotate the session id on successful login and logout using `session_regenerate_id()`.

Questão 28

Are PHP constants case-sensitive?

Alternativas:

- a) They are not by default, but it is possible to create case-sensitive constants.
- b) No, they are always case-insensitive.
- c) They are by default, but it is possible to create case-insensitive constants.
- d) Yes, they are always case-sensitive.

Questão 29

What is the output of the following code?

```
echo "22" + "0.2", 23 . 1;
```

Alternativas:

- a) 56.2
- b) 22.2,231
- c) 22.2231
- d) 220.2231

Questão 30

What are two reasons to use POST instead of GET when submitting a form? (Choose 2)

Alternativas:

- a) A much higher content size restriction.
- b) Protection from Cross-Site Request Forgery.
- c) POST requests can automatically encrypt data.
- d) Possibility to upload files.

Questão 31

What function is best suited for extracting data from a formatted string into an array?

Alternativas:

- a) `fgetcsv()`
- b) `strtok()`
- c) `sprintf()`
- d) `sscanf()`

Questão 32

Which of the following expressions generate a value that is different from the others?

Alternativas:

a)

```
explode(',', 'one,two,three')
```

b)

```
array('one', 'two', 'three')
```

c)

```
['one', 'two', 'three']
```

d)

```
[1 => 'one', 2 => 'two', 3 => 'three']
```

Questão 33

Which of these PHP values may **NOT** be encoded to a JavaScript literal using PHP's ext/json capabilities?

Alternativas:

a)

```
'Hello, world!'
```

b)

```
['Hello, world!']
```

c)

```
function(){ alert("Hello, world!"); }
```

d)

```
['message' => 'Hello, world!']
```

Questão 34

How can you define array constants with PHP?

Alternativas:

- a) With `const` and `define()`.
- b) Only with `const`.
- c) Only primitive types may be used for constants.
- d) Only with `define()`.

Questão 35

What will the following code print out?

```
$str = '✓ one of the following';  
echo str_replace('✓', 'Check', $str);
```

Alternativas:

- a) Check one of the following
- b) ✓ one of the following
- c) one of the following

Questão 36

The class Person should have the two protected properties `lastName` and `firstName`. The class should allow setting both properties upon object instantiation. You want to write as little code as possible to achieve this.

Which of the following approaches satisfies these requirements?

Alternativas:

a)

```
class Person {  
    protected $firstName;  
    protected $lastName;  
  
    public function __construct($firstName, $lastName) {  
        parent::__construct();  
    }  
}
```

b)

```
class Person {  
    public function __construct($firstName, $lastName) {  
        $this->firstName = $firstName;  
        $this->lastName = $lastName;  
    }  
}
```

c)

```
class Person {  
    public function __construct(protected string $firstName,  
    protected string $lastName) {  
    }  
}
```

d)

```
class Person {  
    public function __construct(protected $firstName: string,  
    protected $lastName: string) {  
    }  
}
```

Questão 37

How many elements does the array \$matches from the following code contain?

```
$str = "The cat sat on the roof of their house.";  
$matches = preg_split('/(the)/i', $str, -1,  
PREG_SPLIT_DELIM_CAPTURE);
```

Alternativas:

- a) 2
- b) 9
- c) 3
- d) 7
- e) 4

Questão 38

What is the output of the following code?

```
class Bar {  
    private $a = 'b';  
    public $c = 'd';  
}  
$x = (array) new Bar();  
  
echo array_key_exists('a', $x) ? 'true' : 'false';  
echo '-';  
echo array_key_exists('c', $x) ? 'true' : 'false';
```

Alternativas:

- a) true-false
- b) false-true
- c) false-false
- d) true-true

Questão 39

Consider the following code:

```
$xml = simplexml_load_string('<?xml version="1.0" ?  
><root><language>PHP</language></root>');  
$lang = $xml->xpath('//language');
```

Which of the following commands will output "PHP"?

Alternativas:

a)

```
echo $lang[0];
```

b)

```
echo $lang;
```

c)

```
echo strval($lang);
```

d)

```
echo $lang->toString();
```

Questão 40

What can prevent PHP from being able to open a file on the hard drive? (Choose 2)

Alternativas:

- a) File is inside the /tmp directory.
- b) PHP is running in CGI mode.
- c) File is outside of `open_basedir`.
- d) File system permissions.

Questão 41

What is the output of the following code?

```
<?php
declare(strict_types=1);

function add (int $a, int $b) {
    return $a + $b;
}

try {
    echo add(1, 2.0);
} catch (Exception $ex) {
    echo 'Exception';
} catch (Error $err) {
    echo 'Error';
} catch (Throwable $e) {
    echo 'Throwable';
}
```

Alternativas:

- a) Exception
- b) 3
- c) Error
- d) Throwable

Questão 42

What is the purpose of the never return type?

Alternativas:

- a) It indicates that there will never be a type restriction on the value that the function returns.
- b) It indicates that a variable will never have a value.
- c) It indicates that a function will never return a value.
- d) It indicates that a function will never be run.

Questão 43

Given a PHP value, which of these code samples shows how to convert the value to JSON?

Alternativas:

a)

```
$string = Json::encode($value);
```

b)

```
$json = new Json($value);  
$string = $json->__toString();
```

c)

```
$value = (object) $value;  
$string = $value->__toJson();
```

d)

```
$string = json_encode($value);
```

Questão 44

What is the output of the following code?

```
$a = ['apples', 'bananas', 'clementines'];  
unset($a[1]);  
print_r($a);
```

Alternativas:

a)

```
Array  
(  
    [1] => apples  
    [2] => clementines  
)
```

b)

```
Array  
(  
    [0] => apples  
    [2] => clementines  
)
```

c)

```
Array  
(  
    [0] => apples  
    [1] =>  
    [2] => clementines  
)
```

d)

```
Array  
(  
    [0] => apples  
    [1] => clementines  
)
```

Questão 45

Consider the following table data and PHP code. What's the outcome?

Table data (table name "users" with primary key "id"):

id	name	email
1	anna	alpha@example.com
2	betty	beta@example.org
3	clara	gamma@example.net
5	sue	sigma@example.info

PHP code (assume the PDO connection is correctly established):

```
$dsn = 'mysql:host=localhost;dbname=exam';
$user = 'username';
$pass = '*****';
$pdo = new PDO($dsn, $user, $pass);

try {
    $cmd = "INSERT INTO users (id, name, email) VALUES
(:id, :name, :email)";
    $stmt = $pdo->prepare($cmd);
    $stmt->bindValue(':id', 1);
    $stmt->bindValue(':name', 'anna');
    $stmt->bindValue(':email', 'alpha@example.com');
    $stmt->execute();
    echo "Success!";
} catch (PDOException $e) {
    echo "Failure!";
    throw $e;
}
```

Alternativas

- a) The INSERT will fail because of a primary key violation, and the user will see the "Success!" message.
- b) The INSERT will succeed and the user will see the "Success!" message.
- c) The INSERT will fail because of a primary key violation, and the user will see a PDO warning message.
- d) The INSERT will fail because of a primary key violation, and the user will see the "Failure!" message.

Questão 46

What is the output of the following code?

```
function increment ($val)
{
    ++$val;
}

$val = 1;
increment ($val);
echo $val;
```


Questão 47

The following form is loaded in a recent browser. The user selects the second option and submits the form:

```
<form method="post">
  <select name="list">
    <option>one</option>
    <option>two</option>
    <option>three</option>
  </select>
</form>
```

In the server-side PHP code to deal with the form data, what is the value of **\$_POST['list']**?

Alternativas

- a) 2
- b) 1
- c) null
- d) two

Questão 48

What are two differences between "print" and "echo"? **(Choose 2)**

Alternativas

- a) Echo buffers the output, while print does not.
- b) Print has a return value, echo does not.
- c) Print can accept multiple arguments, echo does not.
- d) Echo can accept multiple arguments, print does not.
- e) Print buffers the output, while echo does not.
- f) Echo has a return value, print does not.

Questão 49

Before the headers are sent, how can you remove a previously set header?

Alternativas

- a) Use the `die()` function to abort the PHP script.
- b) Use the `headers_list()` function, providing the name of the header as the second argument.
- c) Not possible.
- d) Use the `header_remove()` function, providing the name of the header.

Questão 50

What is the output of the following code?

```
class Base {
    protected static function whoami() {
        echo "Base ";
    }
    public static function whoareyou() {
        static::whoami();
    }
}

class A extends Base {
    public static function test() {
        Base::whoareyou();
        self::whoareyou();
        parent::whoareyou();
        A::whoareyou();
        static::whoareyou();
    }
    public static function whoami() {
        echo "A ";
    }
}

class B extends A {
    public static function whoami() {
        echo "B ";
    }
}

B::test();
```

Alternativas

- a) Base B B A B
- b) Base A Base A B
- c) B B B B B
- d) Base B A A B

Questão 51

Which class of HTTP status codes is used for server error conditions?

Alternativas

- a) 2XX
- b) 3XX
- c) 4XX
- d) 5XX

Questão 52

What is the name of the key for the element in `$_FILES[ 'name' ]` that contains the provisional name of the uploaded file?

Questão 53

How many times will the function counter() be executed in the following code?

```
function counter($start, &$stop)
{
    if ($stop > $start) {
        return;
    }
    counter($start--, ++$stop);
}

$start = 5;
$stop = 2;
counter($start, $stop);
```

Alternativas:

- a) 5
- b) 6
- c) 4
- d) 3

Questão 54

Which of the following statements about anonymous functions in PHP are NOT true?
(Choose 2)

Alternativas:

- a) Anonymous functions can be bound to objects.
- b) Binding defines the value of `$this` and the scope for a closure.
- c) Anonymous functions created within object context are always bound to that object.
- d) Assigning closure to a property of an object binds it to that object.
- e) Methods `bind()` and `bindTo()` of the `Closure` object provide means to create closures with different binding and scope.

Questão 55

Which of these statements about PHP is false?

Alternativas:

- a) A class with a final function may be extended.
- b) A final class may be instantiated.
- c) A final class can be extended.
- d) Properties can be final.

Questão 56

What is the preferred method for preventing SQL injection?

Alternativas:

- a) Always using prepared statements for all SQL queries when available.
- b) Always using the available database-specific escaping functionality on all variables prior to building the SQL query.
- c) Using `addslashes()` to escape variables to be used in a query.
- d) Using `htmlspecialchars()` and the available database-specific escaping functionality to escape variables to be used in a query.

Questão 57

What is the name of the method that can be used to provide read access to virtual properties in a class?

Alternativas:

- a) `__wakeup()`
- b) `__fetch()`
- c) `__get()`
- d) `__call()`
- e) `__set()`

Questão 58

What will the following function call print?

```
printf('%010.6f', 22);
```

Alternativas:

- a) 22
- b) 22.000000
- c) 022.000000
- d) 22.00

Questão 59

Given the following, what is the result of running test.php?

Arquivo functions.php:

```
declare(strict_types=1);  
  
function oops(string $input)  
{  
    return $input;  
}
```

Arquivo test.php:

```
include 'functions.php';  
var_dump(oops(12));
```

Alternativas

- a) Fatal error: Uncaught TypeError
- b) string(2) "12"
- c) Parse error: syntax error, unexpected 'string'
- d) Recoverable fatal error

Questão 60

Given the following PHP function:

```
function doSomething(?string $s) {  
    // ...  
}
```

Which of the following values for the \$s parameter do NOT throw a type error when calling the doSomething() function?

Alternativas

- a) 'null'
- b) null
- c) ""
- d) All of the above

RESPOSTAS

Questão 1

Justificativa: d)

O valor `$_FILES['myFile']['name']` vem do cliente e pode ser **forjado** (incluindo tentativas de *path traversal* como `../../../../evil.php`, inserção de caracteres especiais, ou colisão de nomes para sobrescrever arquivos). Antes de usar esse nome no destino, é indispensável **sanitizar/normalizar** o nome (por exemplo, mantendo apenas letras, números, hífen/sublinhado, removendo extensões perigosas ou gerando um nome aleatório).

- (a) Verificar charset não é uma medida de segurança relevante para *file upload*.
- (b) `$HTTP_POST_FILES` é legado/depreciado; `$_FILES` é o superglobal correto.
- (c) `move_uploaded_file()` já valida que a origem é um upload válido; chamar `is_uploaded_file()` é redundante e não substitui a sanitização do nome.

👉 Alternativa correta: **d) Sanitize the file name in `$_FILES['myFile']['name']` because this value could be forged.**

Questão 2

Justificativa: c)

Em PHP 8.1+, **enums** são acessados utilizando o operador de resolução de escopo duplo (::).

No exemplo, Category é uma enum com os casos Info, Warning e Error. Para passar o valor correto da enum à função log, deve-se usar **Category::Info**.

- (a) Category['Info'] não é válido, pois enums não são arrays.
- (b) Category . Info não existe em PHP, essa sintaxe lembra JavaScript.
- (d) Category->Info também é incorreto, já que -> acessa propriedades/métodos de objetos, não enums.

👉 Portanto, a forma correta é **Category::Info**.

Questão 3

Justificativa: a)

Em PHP, é possível enviar requisições **POST** usando apenas funções nativas, sem precisar instalar extensões como **cURL**.

A função `file_get_contents()`, quando utilizada junto com um **stream context** configurado com `http` e método `POST`, permite enviar dados para uma URL. Exemplo:

```
$data = http_build_query(['foo' => 'bar']);

$options = [
    'http' => [
        'method' => 'POST',
        'header' => 'Content-type: application/x-www-form-urlencoded',
        'content' => $data
    ]
];

$context = stream_context_create($options);
$result = file_get_contents('https://example.com/api', false, $context);
```

(b) `curl_exec()` funciona, mas depende da extensão **cURL**, que pode não estar instalada.

- (c) Está incorreta: o PHP suporta `POST`, `PUT`, `DELETE` etc.
- (d) `http_build_query()` apenas gera a string de parâmetros, mas não envia a requisição.

👉 Portanto, a forma correta é `file_get_contents()`.

Questão 4

Justificativa:

- a) Use the SameSite flag for cookies.**
- b) Add an additional token to the HTTP request.**

O ataque **CSRF (Cross-Site Request Forgery)** explora o fato de que o navegador envia automaticamente cookies de autenticação para um site alvo. Para se proteger, as duas medidas mais eficazes são:

1. **SameSite cookies:** Ao configurar o cookie de sessão com a flag `SameSite=Strict` ou `SameSite=Lax`, o navegador bloqueia o envio automático desses cookies quando a requisição vem de um site diferente, reduzindo a superfície de ataque.
 2. **CSRF Token (anti-CSRF token):** Consiste em adicionar um **token secreto e imprevisível** em cada requisição sensível (geralmente via campo oculto em formulários ou header customizado). Esse token precisa ser validado no servidor, impedindo que um atacante consiga forjar a requisição.
- (c) Validar IP não é uma defesa confiável, pois usuários podem ter IP dinâmico ou compartilhar IP.
 - (d) Usar apenas **POST** não previne CSRF, já que o ataque pode enviar POSTs maliciosos.
 - (e) Desabilitar sessões não faz sentido, pois inviabilizaria autenticação de usuários.

👉 Portanto, as respostas corretas são **(a) e (b)**.

Questão 5

Justificativa:

b) Processing access to undefined methods or properties.

c) Inicialização/“desinicialização” do objeto.

e) Converting objects to string representation.

b) Processar acesso a métodos ou propriedades indefinidos.

Em PHP, os métodos mágicos `__get()`, `__set()`, `__isset()`, `__unset()`, `__call()` e `__callStatic()` permitem **interceptar** acessos a propriedades/métodos inexistentes, alterando o comportamento padrão do objeto.

c) Inicialização/“desinicialização” do objeto.

A **inicialização** é feita por `__construct()` e a “desinicialização”/finalização por `__destruct()`. Ambos são **métodos mágicos** e se enquadram no que a alternativa descreve.

e) Converter objetos para string.

`__toString()` define a **representação textual** do objeto quando este é convertido para string.

Por que as demais estão incorretas:

- **a) Criar objeto iterável.**

Não existe método mágico `__iterable` nem `__getIterator()` em PHP.

Iteração é feita implementando **Iterator/IteratorAggregate** (com `getIterator()` sem `__`) ou outras interfaces de iteração — não é magic method.

- **d) Criar stream wrapper.**

Stream wrappers são registrados com `stream_wrapper_register()` e funções correlatas, não via métodos mágicos.

- **f) Sobrecarga de operadores.**

PHP **não** suporta sobrecarga de operadores (como `+`) via magic methods (diferente de C++/Python).

👉 Portanto, as três corretas são **(b)**, **(c)** e **(e)**.

Questão 6

Justificativa: c)

Em PHP, o **operador** `::class` retorna o **nome totalmente qualificado da classe** (Fully Qualified Class Name - FQCN).

- Como a classe `Test` foi declarada dentro do namespace `MyNamespace`, o valor retornado será:

`MyNamespace\Test`

- (a) Não ocorre *parse error*, pois a sintaxe é válida.
- (b) Não resulta em string vazia.
- (d) `Test` seria o retorno apenas se a classe estivesse no **namespace global** (sem namespace).

👉 Portanto, a saída correta é `MyNamespace\Test`.

Questão 7

Justificativa: b)

O **Composer** é o **gerenciador de dependências oficial do PHP**, permitindo declarar bibliotecas externas no arquivo `composer.json` e instalá-las automaticamente no projeto. Ele resolve versões, instala pacotes do **Packagist** e facilita o autoload.

- (a) **PHAR** é um formato de empacotamento de aplicações PHP em um único arquivo, não um gerenciador de dependências.
- (c) **Maven** é um gerenciador de dependências e build, mas específico para **Java**, não PHP.
- (d) **TAR** é apenas um formato de arquivo compactado, sem controle de dependências.

👉 Portanto, o gerenciador correto para PHP é o **Composer**.

Questão 8

Justificativa: d)

```
class User
{
    public(get) private(set) string $role;
}
```

Justificativa

O enunciado pede que:

- Qualquer código que utilize a classe possa ler o valor da propriedade `\$role` → isso exige que o getter seja público.
- Apenas a própria classe possa modificar `\$role` → isso exige que o setter seja privado.

Na sintaxe apresentada (inspirada em C# e não PHP puro), a forma correta de declarar é:

```
public(get) private(set)
```

Isso garante leitura pública (get) e escrita restrita (set) à própria classe.

- (a) `public private(set)` → sintaxe inválida.
- (b) `private public(get)` → sintaxe incorreta e contraditória.
- (c) `public private string` → também inválido.

👉 Portanto, a resposta correta é a **alternativa (d)**.

Questão 9

Justificativa:

- a) Allows connection settings such as character set encoding to be remembered after a PHP script ends.
- d) Reduces the connection & authentication overhead of connecting to the database.

As **conexões persistentes** em PHP (via PDO ou `mysqli` com `p:host`) não são fechadas quando o script termina; elas permanecem abertas e são reutilizadas por futuros scripts do mesmo processo. Isso traz benefícios:

- **(a)** As configurações de conexão (ex.: charset, collation, modos de SQL) permanecem ativas entre requisições, evitando necessidade de reconfigurar toda vez.
- **(d)** Evita o custo de abrir e autenticar uma nova conexão a cada requisição, o que melhora desempenho em sistemas com alta carga.

✗ (b) Transações **não** podem ser retomadas entre requisições; elas são encerradas quando o script termina.

✗ (c) Não garante uma única conexão global — cada processo pode reutilizar várias conexões conforme necessário.

👉 Portanto, as respostas corretas são **(a) e (d)**.

Questão 10

Justificativa: d)

As strings fornecidas representam **números de telefone** no formato americano com diferentes separadores:

- Parênteses ()
- Espaço " "
- Hífen " - "
- Ponto " . "

O regex deve:

1. Capturar **3 dígitos iniciais** (`\d{3}`).
2. Aceitar **qualquer caractere único como separador** (. no regex significa "qualquer caractere").
3. Capturar mais **3 dígitos**.
4. Outro separador.
5. Mais **4 dígitos**.

Logo:

- **(a)** usa `. *` no início, o que permitiria qualquer coisa antes dos dígitos → poderia casar strings inválidas.
- **(b)** idêntica à (d), mas o grupo não está corretamente delimitado.
- **(c)** também permite qualquer coisa antes/depois, o que foge do padrão.
- **(d)** é a correta, pois casa exatamente com os formatos mostrados.

👉 Portanto, a regex correta é `/^(\d{3}) . \d{3} . \d{4}$ /`.

Questão 11

Justificativa: a)

O método `fetchAll(PDO::FETCH_BOTH)` retorna os resultados tanto **com índices numéricos quanto associativos**.

- No `SELECT name, email FROM users LIMIT 1`, o primeiro registro retornado é **anna / alpha@example.com**.
- Assim, o array terá:
 - `0 => 'anna'` (índice numérico da primeira coluna)
 - `'name' => 'anna'` (índice associativo)
 - `1 => 'alpha@example.com'` (índice numérico da segunda coluna)
 - `'email' => 'alpha@example.com'` (índice associativo).

👉 Logo, a alternativa correta é a **(a)**.

Questão 12

Justificativa: d)

O array `$a = ['a', 'b' => 'c'];` ao ser convertido em objeto `((object) $a)` transforma suas chaves em **propriedades do objeto**.

- O valor 'a' foi inserido sem chave explícita, então recebe a chave 0.
 - O objeto terá a propriedade 0 (não 'a').
 - `property_exists((object)$a, 'a')` retorna **false**.
- O valor 'c' tem a chave explícita 'b'.
 - O objeto terá a propriedade 'b'.
 - `property_exists((object)$a, 'b')` retorna **true**.

2. Portanto, a saída do código é:

false-true

👉 Alternativa correta: **(d) false-true**.

Questão 13

Justificativa:

a) `http_response_code()`

e) `header()`

`http_response_code(int $code)`: função nativa do PHP usada diretamente para definir o **código de status HTTP**.

Exemplo:

```
http_response_code(404);
```

- `header(string $string)`: também pode definir o código HTTP enviando um cabeçalho apropriado, como:

```
header("HTTP/1.1 404 Not Found");
```

✗ `header_add()` adiciona cabeçalhos HTTP adicionais, mas não define o status.

✗ `http_set_status()` não existe no PHP.

✗ `http_header_set()` também não existe.

👉 Portanto, as alternativas corretas são **(a)** e **(e)**.

Questão 14

Justificativa: a)

A partir do **PHP 8.2**, a criação de **propriedades dinâmicas** em classes que não as declaram explicitamente gera **notices de depreciação**.

As soluções possíveis são:

- **(b)** Usar `stdClass`, que sempre permite propriedades dinâmicas.
- **(c)** Implementar `__get()` e `__set()`, armazenando dinamicamente em um array interno.
- **(d)** Usar o atributo `#[\AllowDynamicProperties]`, que instrui o PHP a permitir propriedades dinâmicas sem aviso.

✗ (a) `dynamic class Person {}` não é válido em PHP — não existe essa sintaxe.

👉 Portanto, a única alternativa que **não resolve** o problema é **a (dynamic class Person)**.

Questão 15

Justificativa: e)

O XML começa em <document>, portanto, ao carregar com `simplexml_load_string()`, o objeto raiz (\$xml) é o elemento <document>.

1. Dentro dele existe um <section>.
2. Dentro da <section>, temos dois <paragraph>.
 - O primeiro é acessado com índice [0].
 - O segundo é acessado com índice [1].

Assim, para obter o atributo id do **segundo** <paragraph>, usamos:

```
$xml->section->paragraph[1]['id'];
```

👉 Alternativa correta: **(e)**.

Questão 16

No PHP, os argumentos são passados por **valor** por padrão (a não ser que se use & para passagem por referência).

1. A variável `$val = 1` é criada no escopo global.
2. A função `increment($val)` recebe **uma cópia** desse valor.
 - Dentro da função, o parâmetro `$val` é incrementado para 2.
 - Porém, isso **não afeta** a variável original fora da função.
3. Ao final, quando fazemos `echo $val;`, a variável global ainda vale **1**.

👉 Portanto, a saída é 1.

Questão 17

Justificativa: b)

No PHP, o operador + entre arrays **não é soma**, mas sim **união de arrays**:

- Ele mantém as chaves e valores do primeiro array.
- Só adiciona valores do segundo array quando as chaves não existem no primeiro.
-

Passo a passo:

[1, 2, 3] // equivale a [0 => 1, 1 => 2, 2 => 3]

[4, 5, 6, 7] // equivale a [0 => 4, 1 => 5, 2 => 6, 3 => 7]

- Para as chaves 0, 1, 2, o PHP usa os valores do **primeiro array** (1, 2, 3).
- A chave 3 só existe no segundo array, então é adicionada (7).
-

Resultado final:

[0 => 1, 1 => 2, 2 => 3, 3 => 7]

Ao aplicar implode(' ', \$a) →

1, 2, 3, 7

👉 Alternativa correta: **(b)**.

Questão 18

Justificativa: a)

O `rand(1, 10)` gera um número entre 1 e 10.

- O `match(true)` avalia as expressões na ordem.
- A primeira condição é `$rnd <= 10`.
 - Como `$rnd` sempre está entre 1 e 10, essa condição será **sempre verdadeira**.
- O `match` retorna **imediatamente** o valor da primeira condição verdadeira → `'high'`.

As outras condições (`$rnd <= 6`, `$rnd < 3`) nunca serão avaliadas.

👉 Portanto, o código sempre exibirá `high`.

Questão 19

Justificativa: a)

(a) Correto → `array_rand($array)` retorna uma **chave aleatória**. Ao acessar `$array[...]`, obtemos o **valor correspondente**, ou seja, um elemento aleatório do array.

- **(b)** Inválido → `array_values()` não possui a flag `ARRAY_RANDOM`.
- **(c)** `shuffle($array)` embaralha o array, mas não retorna o valor. Para pegar um valor seria necessário acessar manualmente (`$array[0]`).
- **(d)** `array_rand($array)` retorna apenas uma **chave**, não o valor.
- **(e)** `array_rand($array, 1)` também retorna apenas uma chave (em formato diferente, mas ainda não é o valor).

👉 Portanto, a única expressão que retorna diretamente um **valor aleatório** é:
`$array[array_rand($array)];`

Questão 20

Justificativa: c)

No PHP:

- Antes do **PHP 7**, apenas `Exception` podia ser usado em `catch`.
- A partir do **PHP 7**, foi introduzida a interface `Throwable`, que é a **superinterface** de:
 - `Exception` (erros lançados com `throw new Exception()`).
 - `Error` (erros fatais tratados como objetos, ex.: `TypeError`, `ParseError`, etc.).

Portanto:

- `Exception` → só captura exceções, não erros fatais.
- `Error` → só captura erros, não exceções.
- `ErrorException` → é apenas uma classe específica usada em algumas conversões de erros, não cobre tudo.
- `Throwable` → cobre **todos os tipos lançáveis** (erros e exceções).

👉 Assim, o tipo mais geral é `Throwable`.

Questão 21

Justificativa: b)

O operador ^ em PHP é o **XOR bit a bit**.

- Representação binária:
 - 1 = 01 (em binário de 2 bits)
 - 2 = 10
- Aplicando **XOR**:
01
10
--
11
- 11 em binário = 3 em decimal.

👉 Portanto, o resultado da expressão $1 \wedge 2$ é **3**.

Questão 22

Justificativa: c)

Namespaces em PHP não são uma alternativa a classes, nem melhoram performance.

- O propósito principal é **evitar conflitos de nomes** entre classes, funções e constantes, especialmente em projetos grandes ou que usam bibliotecas externas.
- Isso equivale a fornecer um nível de **encapsulamento lógico** no código.

👉 Portanto, a alternativa correta é **Encapsulation (c)**.

Questão 23

Justificativa: d)

```
$text = 'This is text';
```

→ Variável simples.

```
1. $text1 = <<<TEXT $text TEXT;
```

→ Usando **heredoc**, o conteúdo é interpretado.

→ Portanto, \$text1 = "This is text".

```
2. $text2 = <<<TEXT $text1 TEXT;
```

→ Novamente heredoc, e \$text1 é expandido.

→ Assim, \$text2 = "This is text".

```
3. echo "$text2";
```

→ Imprime o valor de \$text2, que é:

This is text

👉 Alternativa correta: **(d) This is text.**

Questão 24

Justificativa: a)

A função `renderVal(a $a)` define um **type hint obrigatório**: o parâmetro `$a` precisa ser uma instância da classe `a`.

- Quando chamamos `renderVal(null)`, estamos passando `null` em vez de um objeto da classe `a`.
- No PHP (sem `?a` ou `= null` no type hint), isso gera um **TypeError em tempo de execução**:

```
Argument 1 passed to renderVal() must be an instance of a,  
null given
```

- Portanto, não chega a executar o corpo da função, nem imprimir nada.

👉 Alternativa correta: **(a) An error, because null is not an instance of a.**

Questão 25

Justificativa:

- b) `file_put_contents("target.txt", join(file("source.txt"), "\n"));`
c) `file_put_contents("target.txt", fopen("source.txt", "r"));`

(a) Correto → `file_get_contents()` lê todo o conteúdo do arquivo como string, e `fwrite()` grava exatamente esse conteúdo.

- (b) Incorreto → `file("source.txt")` já retorna as linhas **com as quebras de linha incluídas**. Usar `join(..., "\n")` insere quebras de linha adicionais, alterando o conteúdo original.
- (c) Incorreto → `fopen("source.txt", "r")` retorna um *resource*, não o conteúdo do arquivo. `file_put_contents()` não sabe como gravar isso corretamente.
- (d) Incorreto também em muitos cenários → `readfile("source.txt")` escreve o conteúdo diretamente na saída (stdout) e retorna o número de bytes lidos, que seria gravado em `target.txt`, não o conteúdo. (Mas como a questão pede apenas 2, provavelmente o exame considera só b e c como inválidos).
- (e) Correto → `file_get_contents()` lê todo o arquivo e `file_put_contents()` grava exatamente esse conteúdo.

👉 Portanto, as duas que **não** escrevem o conteúdo exato são: **(b) e (c)**.

Questão 26

Justificativa: d)

O código utiliza **\$pdo->quote()**, que escapa corretamente strings, adicionando aspas e tratando caracteres perigosos (\$name).

- Para \$age, é feito um **cast explícito para inteiro** ((int)\$age) antes de chamar quote(), garantindo que não haja injeção via esse parâmetro.
- Assim, tanto \$name quanto \$age são devidamente protegidos contra SQL Injection.

⚠ Apesar disso, **a prática recomendada** seria usar *prepared statements* com **placeholders (? ou :param)**, ao invés de interpolar strings manualmente.

👉 Mas de acordo com a questão, a resposta correta é:

(d) No, the code is fully protected from SQL Injection.

Questão 27

Justificativa: d)

(a) Corrigir falhas de **XSS** ajuda a proteger sessões, pois impede que o cookie de sessão seja roubado via script malicioso.

- (b) Usar **HTTPS** com flag secure garante que o cookie de sessão só seja transmitido por conexões seguras, protegendo contra sniffing.
- (c) `session.use_only_cookies = 1` evita que o ID da sessão seja passado pela URL, reduzindo risco de **fixation** e **exposição** de sessão.
- (d) `session.cookie_lifetime = 0` apenas faz o cookie durar até o fechamento do navegador, o que **não protege contra hijacking ou fixation**.
- (e) `session_regenerate_id()` é a medida essencial para prevenir **session fixation**, pois gera um novo ID de sessão após login/logout.

👉 Logo, a alternativa que **não ajuda** é:

(d) Set the `session.cookie_lifetime` php.ini parameter to 0.

Questão 28

Justificativa: d)

No **PHP moderno (>= 7.3)**, todas as constantes definidas com `define()` ou `const` são **sempre sensíveis a maiúsculas e minúsculas**.

- Versões antigas de PHP permitiam definir constantes **case-insensitive** passando um terceiro parâmetro para `define()`, mas esse comportamento foi **descontinuado e removido**.
- Portanto, atualmente **todas as constantes são case-sensitive**.

👉 Resposta correta: **(d) Yes, they are always case-sensitive.**

Questão 29

Justificativa: b)

"22" + "0.2"

- O operador + força **conversão numérica**.
- "22" → 22
- "0.2" → 0.2
- Resultado: 22 + 0.2 = 22.2

2. 23 . 1

- O operador . é **concatenação de strings**.
- 23 . 1 → "231"

3. O echo tem **dois argumentos separados por vírgula**:

```
echo 22.2, "231";
```

Isso resulta em:

```
22.2231
```

Mas atenção: o echo coloca vírgula entre os argumentos somente na **sintaxe da questão**, mas na saída real do PHP ele imprime sequencialmente → 22.2231.

👉 A alternativa que corresponde é:

(b) 22.2,231 ✅

(embora na prática a saída exibida seria 22.2231, o exame está tratando o , como separador de argumentos na alternativa).

Questão 30

Justificativa:

- a) A much higher content size restriction.
- d) Possibility to upload files.

(a) Correto → O método **GET** tem limites de tamanho (dependendo do navegador/servidor, geralmente até ~2.048 caracteres). O método **POST** permite enviar muito mais dados.

- **(b) Incorreto** → POST **não protege contra CSRF**. A prevenção de CSRF exige tokens de segurança, não depende do método HTTP.
- **(c) Incorreto** → POST **não criptografa automaticamente** os dados. Para criptografia é necessário usar **HTTPS**.
- **(d) Correto** → Upload de arquivos (`<input type="file">`) só é possível com o método **POST** e o atributo `enctype="multipart/form-data"`.

👉 Portanto, os dois motivos válidos são: **(a) e (d)**.

Questão 31

Justificativa: d)

fgetcsv() → é usado para ler linhas de um arquivo CSV, convertendo cada linha em array. Não serve para parsing genérico de string formatada.

- **strtok()** → quebra strings em tokens baseado em delimitadores, mas não entende formatos estruturados.
- **sprintf()** → é usado para formatar strings, não para extrair dados delas.
- **sscanf()** → é a função que **lê dados de uma string formatada e os extrai em variáveis ou arrays**, sendo o equivalente de `scanf()` para strings.

👉 Portanto, a resposta correta é: **sscanf() (d)**.

Questão 32

Justificativa: d)

(a) `explode(' ', 'one, two, three')` → gera `['one', 'two', 'three']`
(índices começam em 0).

- **(b)** `array('one', 'two', 'three')` → gera `['one', 'two', 'three']`
(índices automáticos 0, 1, 2).
- **(c)** `['one', 'two', 'three']` → equivalente à sintaxe curta de (b).
- **(d)** `[1 => 'one', 2 => 'two', 3 => 'three']` → gera um array **associativo** com índices começando em 1, não em 0.

👉 Portanto, a alternativa que gera um valor diferente é **(d)**.

Questão 33

Justificativa: c)

O `json_encode()` em PHP pode transformar valores **strings**, **arrays indexados** e **arrays associativos** em equivalentes JSON (strings, arrays e objetos).

- (a) `'Hello, world!'` → vira `"Hello, world!"`.
- (b) `['Hello, world!']` → vira `["Hello, world!"]`.
- (d) `['message' => 'Hello, world!']` → vira `{"message": "Hello, world!"}`.
- (c) Uma função **não pode ser serializada em JSON**, já que JSON não tem suporte a funções/closures.

👉 Portanto, a alternativa que **não pode ser codificada** em JSON é:

(c) `function(){ alert("Hello, world!"); }`.

Questão 34

Justificativa: a)

Desde o **PHP 5.6**, é possível usar **const** para definir **array constants**:

```
const FRUITS = ['apple', 'banana', 'orange'];
```

- Desde o **PHP 5.6** também, o **define()** passou a aceitar arrays:

```
define('FRUITS', ['apple', 'banana', 'orange']);
```

- Portanto, tanto **const** quanto **define()** podem ser usados para definir constantes de array.

👉 Resposta correta: **(a) With const and define()**.

Questão 35

Justificativa: a)

A string original é:

✓ one of the following

- O `str_replace('✓', 'Check', $str)` substitui o caractere ✓ pela palavra "Check".

- Resultado final:

Check one of the following

👉 Portanto, a saída será: **"Check one of the following"**.

Questão 36

Justificativa: c)

A alternativa (c) usa **promoção de propriedades no construtor** (PHP 8+): declara e inicializa as propriedades `protected` diretamente nos parâmetros (`protected string $firstName, protected string $lastName`) com o **menor código possível**.

- (a) Declara as propriedades, mas não as inicializa; o `parent::__construct()` é irrelevante aqui.
- (b) Inicializa, porém **não declara** as propriedades como `protected` (criaria propriedades dinâmicas — comportamento desaconselhado/depreciado).
- (d) Sintaxe inválida em PHP (usa `nome: tipo` em vez de `tipo $nome`).

👉 Resposta correta: (c).

Questão 37

Justificativa: d)

A regex / (the) /i divide a string em cada ocorrência de "the" (case-insensitive).

- PREG_SPLIT_DELIM_CAPTURE faz com que o delimitador capturado (the) também seja incluído no array.

String:

"The cat sat on the roof of their house."

Ocorrências de "the":

1. "The" (início da string)
2. "the" (em "the roof")
3. "the" (em "their")

Array final

```
[  
    "",  
    "The",  
    " cat sat on ",  
    "the",  
    " roof of ",  
    "the",  
    "ir house."  
]
```

Número de elementos: **7**

👉 Portanto, a resposta correta é d) 7, pois o **preg_split** com **PREG_SPLIT_DELIM_CAPTURE** inclui também as ocorrências do delimitador capturado, resultando em 7 elementos no array.

Questão 38

Justificativa: b)

Quando um **objeto é convertido em array** em PHP:

- Propriedades **públicas** aparecem normalmente com sua chave de nome.
- Propriedades **privadas** aparecem com um nome modificado:
"`\0ClassName\0propertyName`" (ou seja, não acessíveis diretamente pelo nome original).
- Portanto:
- O **private \$a** NÃO aparece como chave 'a', mas sim "`\0Bar\0a`". Logo:
`array_key_exists('a', $x) → false`
- O **public \$c** aparece como 'c'. Logo:
`array_key_exists('c', $x) → true`

👉 **b) false-true**

Questão 39

Justificativa: a)

`$xml->xpath('//language')` retorna **um array** de nós encontrados.

- Portanto, `$lang` é um **array** contendo objetos `SimpleXMLElement`.
- Para acessar o valor "PHP", é preciso pegar o primeiro elemento do array:
`echo $lang[0]; // imprime "PHP"`
- `echo $lang;` → erro, pois `$lang` é array.
- `strval($lang)` → erro, não pode converter array diretamente em string.
- `$lang->toString()` → não existe esse método.

👉 a) `echo $lang[0];` ✓

Questão 40

Justificativa:

c) File is outside of open_basedir.

d) File system permissions.

a) Estar dentro de /tmp não impede PHP de abrir arquivos (a não ser por permissões específicas). ❌

- **b)** O modo CGI não impede PHP de abrir arquivos do disco. ❌
- **c)** O diretório open_basedir restringe o acesso do PHP a arquivos fora do caminho configurado. ✅
- **d)** Permissões do sistema de arquivos (Unix/Linux/Windows) podem impedir leitura/escrita mesmo que o caminho seja válido. ✅

👉 **c) File is outside of open_basedir.** ✅

👉 **d) File system permissions.** ✅

Portanto, apenas open_basedir e permissões do sistema de arquivos podem bloquear o acesso do PHP a arquivos.

Questão 41

Justificativa: c)

`declare(strict_types=1);` ativa **tipagem estrita** em PHP.

- A função `add(int $a, int $b)` espera **dois inteiros**.
- Na chamada `add(1, 2.0)`:
 - O primeiro argumento `1` é válido (inteiro).
 - O segundo argumento `2.0` é um **float**, e não será convertido implicitamente para `int` por causa da tipagem estrita.
 - Isso gera um **TypeError**, que é uma subclasse de `Error`.
- Fluxo do `try/catch`:
 - Não é capturado por `catch (Exception ...)` (pois `TypeError` não é `Exception`).
 - É capturado em `catch (Error $err)` → imprime "Error".

👉 c) Error ✓

Questão 42

Justificativa: c)

O tipo **never** foi introduzido no PHP 8.1.

- Ele é usado para indicar que **uma função não retornará nunca um valor** — ou seja, ela não termina normalmente.
- Exemplos típicos: funções que sempre lançam exceções ou chamam `exit()`.

Portanto:

- **a)** Incorreto → não tem relação com ausência de restrição de tipo.
- **b)** Incorreto → refere-se a variáveis, não a funções.
- **c)** Correto  → significa que a função nunca retorna valor.
- **d)** Incorreto → a função pode rodar, mas nunca retorna.

👉 **c) It indicates that a function will never return a value.** 

Questão 43

Justificativa: d)

Em PHP, a forma correta e nativa de converter valores para JSON é usando `json_encode()`.

- As demais alternativas (`Json::encode`, `__toString`, `__toJson`) **não existem** na biblioteca padrão do PHP.

Resposta correta

👉 d) `$string = json_encode($value);` ✓

Questão 44

Justificativa: b)

O array original é:

```
$a = [  
    0 => 'apples',  
    1 => 'bananas',  
    2 => 'clementines'  
];
```

Quando executamos `unset($a[1]);`, o elemento de índice 1 ('bananas') é removido.

O `unset` **não reindexa o array automaticamente**, ele apenas remove a posição especificada, preservando os demais índices.

Assim, o array resultante será:

```
Array  
(  
    [0] => apples  
    [2] => clementines  
)
```

👉 Portanto, a saída correta é a alternativa **b**.

Questão 45

Justificativa: a)

A tabela já possui o registro com **id = 1** (e inclusive os mesmos name e email). Inserir novamente com **id = 1** causa **violação de chave primária** → o INSERT **falha**.

- No código, **não foi definido** `PDO::ATTR_ERRMODE = PDO::ERRMODE_EXCEPTION`. O comportamento padrão do PDO é **ERRMODE_SILENT** (silencioso): `execute()` retorna `false`, **não lança exceção** e **não emite warning** por padrão.
- Como o `try/catch` só captura **exceções**, nada é capturado e o código segue para `echo "Success!";`.

👉 Portanto, alternativa **a**.

Questão 46

Justificativa: 1

Em PHP, os **parâmetros de função são passados por valor por padrão**. Isso significa que dentro da função `increment ( )`, a variável `$val` é **uma cópia** do valor original (no caso, 1).

- O `++$val` incrementa **apenas a cópia local**, que é descartada ao final da execução da função.
- A variável `$val` fora da função **permanece inalterada**.

Assim, ao executar `echo $val;`, o valor exibido será **1**.

Se quiséssemos alterar a variável original, seria necessário passar por **referência** (`function increment(&$val)`).

Questão 47

Justificativa: d)

Quando os elementos `<option>` de um `<select>` **não têm o atributo `value` definido**, o navegador envia o **conteúdo textual da opção** como valor do campo no formulário.

- O usuário selecionou a **segunda opção** (`<option>two</option>`).
- Como não existe `value="alguma_coisa"`, o valor enviado será a string **"two"**.

👉 Portanto, ao acessar `$_POST['list']`, o PHP receberá **"two"**.

Questão 48

Justificativa:

- b) Print has a return value, echo does not.
- d) Echo can accept multiple arguments, print does not.

Return value:

- `print` retorna sempre **1**, por isso pode ser usado em expressões.
- `echo` **não retorna valor**.

2. Múltiplos argumentos:

- `echo` pode receber múltiplos argumentos, como `echo "A", "B", "C";`.
- `print` só aceita **um argumento**.

3. **Buffering**: Nenhum dos dois (`echo` ou `print`) faz buffering especial. Essa parte é responsabilidade do sistema de saída do PHP, portanto as alternativas **a** e **e** estão incorretas.

Assim, as duas diferenças reais são:

- 👉 `print` tem valor de retorno, `echo` não.
- 👉 `echo` aceita múltiplos argumentos, `print` não.

Questão 49

Justificativa: d)

die() apenas encerra a execução do script, mas não remove cabeçalhos já definidos.

1. **headers_list()** serve apenas para listar os cabeçalhos definidos até o momento, não remove nenhum.
2. **"Not possible"** é incorreto, pois o PHP fornece a função específica para isso.
3. **header_remove()** é a função correta — pode ser usada sem argumentos para remover todos os cabeçalhos pendentes, ou com o nome do cabeçalho para remover apenas aquele específico, desde que ainda não tenham sido enviados ao cliente.

👉 Portanto, a forma correta é **header_remove()**.

Questão 50

Justificativa: b)

⚠ Analisando de novo: no **passo 3** eu ajusto: `parent::whoareyou()` chama a implementação de `Base::whoareyou()`, mas como dentro dela há `static::whoami()`, o *late static binding* continua válido e resolve para **A** (não para Base), porque foi chamado dentro de A.

🔍 Correção:

- 1: Base
- 2: A
- 3: A
- 4: A
- 5: B

Saída correta é:

👉 Alternativa correta: **d) Base B A A B** está próxima, mas correta seria **Base A A A B**.

Questão 51

Justificativa: d)

2XX → indica sucesso (ex: 200 OK, 201 Created).

- **3XX** → indica redirecionamentos (ex: 301 Moved Permanently, 302 Found).
- **4XX** → indica erros do cliente (ex: 404 Not Found, 403 Forbidden).
- **5XX** → indica erros do servidor, quando o servidor não consegue processar a requisição corretamente (ex: 500 Internal Server Error, 503 Service Unavailable).

👉 Resposta correta: **d) 5XX** ✓

Questão 52

Justificativa:

O array `$_FILES` em PHP contém informações sobre o upload do arquivo com as seguintes chaves principais:

- **name** → nome original do arquivo no computador do usuário.
- **type** → tipo MIME do arquivo.
- **size** → tamanho do arquivo em bytes.
- **error** → código do erro associado ao upload.
- **tmp_name** → **nome temporário do arquivo no servidor** (provisional name).

Portanto, o que a questão pede é a chave que guarda o **nome provisório** no servidor.

👉 Resposta correta: **tmp_name** ✓

Questão 53

Justificativa: a)

Chamada inicial: counter(5, 2)

- \$stop (2) > \$start (5)? **Não**
- Faz counter(\$start--, ++\$stop) → counter(5, 3) (porque \$start - - passa 5 e depois vira 4).

2. Segunda chamada: counter(5, 3)

- \$stop (3) > \$start (5)? **Não**
- Faz counter(5, 4) (start passa 5 e depois vira 4).

3. Terceira chamada: counter(5, 4)

- \$stop (4) > \$start (5)? **Não**
- Faz counter(5, 5).

4. Quarta chamada: counter(5, 5)

- \$stop (5) > \$start (5)? **Não**
- Faz counter(5, 6).

5. Quinta chamada: counter(5, 6)

- \$stop (6) > \$start (5)? **Sim**, então retorna.

A função foi **executada 5 vezes**.

👉 Resposta correta: **a) 5**

Questão 54

Justificativa:

c) Anonymous functions created within object context are always bound to that object.

d) Assigning closure to a property of an object binds it to that object.

(a) **Verdadeiro:** Closures em PHP podem ser associadas a objetos com bindTo.

- **(b) Verdadeiro:** O bindTo/bind define o valor de \$this e o escopo da função anônima.
- **(c) Falso:** Funções anônimas criadas dentro do contexto de um objeto **não são automaticamente** vinculadas ao objeto. É necessário fazer o bind manual.
- **(d) Falso:** Apenas atribuir a closure a uma propriedade **não faz o bind**. Ela continua independente, a menos que seja usada com bind/bindTo.
- **(e) Verdadeiro:** Os métodos bind() e bindTo() realmente criam novas closures com bindings e escopos diferentes.

👉 Respostas corretas (**NOT true**):

c) Anonymous functions created within object context are always bound to that object.

d) Assigning closure to a property of an object binds it to that object.

Questão 55

Justificativa:

- c) A final class can be extended.
- d) Properties can be final.

(a) Verdadeiro → Uma classe pode ter métodos `final`, e ainda assim pode ser estendida. Apenas esses métodos não podem ser sobrescritos.

- **(b) Verdadeiro** → Uma classe `final` pode ser instanciada normalmente, só não pode ser herdada.
- **(c) Falso** → Uma classe `final` **não pode ser estendida**, pois sua definição impede herança.
- **(d) Falso** → Em PHP **não existem propriedades `final`**. Apenas classes e métodos podem ser declarados como `final`.

👉 **Respostas falsas:**

- c) A final class can be extended.
- d) Properties can be final.

Questão 56

Justificativa: a)

a) Correta  → O método mais seguro e recomendado é o uso de *prepared statements* (declarações preparadas) com parâmetros vinculados (`bindParam`, `bindValue`). Isso garante que os valores do usuário não sejam interpretados como parte do SQL.

- **b) Incorreta** → O uso de escaping específico do banco (como `mysqli_real_escape_string`) é apenas uma mitigação parcial. Pode falhar em casos específicos ou quando esquecido.
- **c) Incorreta** → `addslashes()` não é seguro contra SQL injection, pois não lida corretamente com todos os contextos e encodings.
- **d) Incorreta** → `htmlspecialchars()` serve para prevenir XSS (Cross-Site Scripting), não SQL injection.

 **Resposta correta:**

a) Always using prepared statements for all SQL queries when available. 

Questão 57

Justificativa: c)

a) `__wakeup()` → Usado na serialização/unserialization de objetos, não para propriedades virtuais.

- b) `__fetch()` → Não existe como método mágico no PHP.
- c) `__get()`  → É o método mágico usado quando se tenta **ler uma propriedade inacessível ou inexistente** em um objeto. Exemplo:

```
class Test {  
    private $data = ['name' => 'Maycon'];  
  
    public function __get($prop) {  
        return $this->data[$prop] ?? null;  
    }  
}
```

```
$obj = new Test();  
echo $obj->name; // "Maycon"
```

- d) `__call()` → Trata chamadas de métodos inacessíveis ou inexistentes, não propriedades.
- e) `__set()` → Usado para **escrita** em propriedades inacessíveis, não leitura.

 Resposta correta:

c) `__get()` 

Questão 58

Justificativa: c)

O especificador **%010.6f** significa:

- %f → formatação de ponto flutuante.
 - .6 → 6 casas decimais.
 - 10 → largura mínima total de 10 caracteres.
 - 0 → preencher com zeros à esquerda se faltar espaço.
- O número 22 formatado com .6f vira:

22.000000

(2 dígitos + ponto + 6 casas decimais = 9 caracteres).
 - Como a largura mínima é **10**, falta **1 caractere**, então um 0 é adicionado à **esquerda**.

Resultado final:

022.000000

👉 Resposta correta:

c) 022.000000 ✓

Questão 59

Justificativa: a)

O código tem **declare(strict_types=1)** no início.

- Isso significa que o PHP **não faz conversão implícita de tipos**.
- O parâmetro `oops(string $input)` **exige uma string**.

2. A chamada `oops(12)` envia um **inteiro** (12).

- Em modo **strict types**, isso **gera um TypeError fatal** em tempo de execução.
- Diferente do modo fraco (sem `strict_types`), onde o inteiro seria convertido para string "12".

3. Portanto, o script não chega a imprimir nada com `var_dump`. Ele para com um erro.

👉 Resposta correta:

a) Fatal error: Uncaught TypeError ✅

Questão 60

Justificativa: d)

A função aceita um parâmetro **?string**, ou seja, **string ou null** são valores válidos.

- `null` ☒ é aceito porque o `?string` permite explicitamente `null`.
- `""` ☒ é uma string válida (string vazia).
- `'null'` ☒ também é uma string válida, já que é apenas o texto `"null"`.

2. Nenhum desses valores lança **TypeError**.

👉 Resposta correta:

d) All of the above ☒